Deep Dive 07 | Frontend Architecture and Deployment

Trace React composition, SPA routing, environment configuration and production hosting.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. React application structure

main.jsx is the client entry; App.jsx composes the route table and shared footer. Pages live under frontend/src/app and components under app/components. AuthContext shares user state and auth operations; ToastContext supports feedback. The project keeps a Next-like folder naming convention in places, but it is a Vite React application, not a Next.js App Router runtime.

2. Navigation and route fallback

React Router maps paths to page components. Because routes are resolved in the browser, the production host must return index.html for direct loads and refreshes of app paths. The Vercel rewrite config sends /api and /uploads to Render, then routes other paths to index.html. Express also has a static build and HTML fallback path.

3. Loading and effects

- NearbyMap is lazy-loaded and wrapped with Suspense so Leaflet code loads only when the page needs it.
- AuthContext reads localStorage during initialization and updates it after login/register/refresh.
- AlertTicker loads summaries on mount and polls on an interval; it clears the interval on unmount.
- Location effects request browser permission and select a fallback when permission fails.
- Effect cleanup and stale async responses should be reviewed when users navigate quickly.

4. Styling and component concerns

The main web app uses Tailwind CSS and custom global styles. Some app components also contain inline colors/layout styles. Reusable design tokens and component variants would improve consistency. The standalone earthquake quiz has a separate Vite config and dependency set; it should not be assumed to share the main app build.

5. Deployment checklist

Service	Configuration responsibilities
Vercel	Frontend root, npm build, dist output, SPA rewrite, /api and /uploads rewrites.
Render Express	Node start command, PORT, MONGO_URI, JWT_SECRET, SESSION_SECRET, FRONTEND_URL, OAuth and Geoapify keys.
Flask service	Python requirements, gunicorn entry point, GEMINI_API_KEY, CORS, stable service URL.
Local Vite	Port 5173, API proxy to localhost:5000, VITE_FLASK_URL for chatbot.
Express single-service option	Build frontend first; Express serves frontend/dist and API routes.

6. Common deployment failures

- Frontend bundle contains an old VITE_* value because the build environment was not updated/rebuilt.
- Vercel rewrite points to an old Render hostname.
- Google OAuth callback URL or FRONTEND_URL differs from local/deployed origins.
- Render environment secret is missing or contains a placeholder.

- Direct deep-route refresh returns 404 because SPA fallback is absent.
- Flask service URL is unreachable or cross-origin configuration is incorrect.

7. Interview focus

Explain how a Vite app differs from Next.js: Vite builds a client-side React bundle, while Next.js can provide framework-managed routing/rendering/server features. In Ready Scout, React Router owns client routes and Express owns the API; static hosting plus rewrites replace the previous Next server responsibilities.